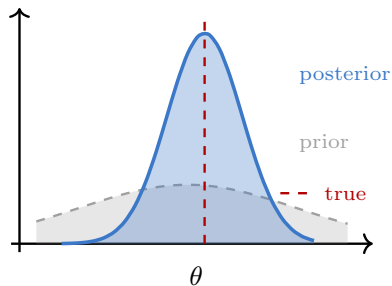


InferCell: Inference-First Whole-Cell Modelling

Tom Kimpson

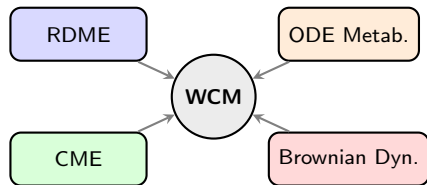
University of Melbourne

MACSYS Research Theme 1 Workshop, May 2026

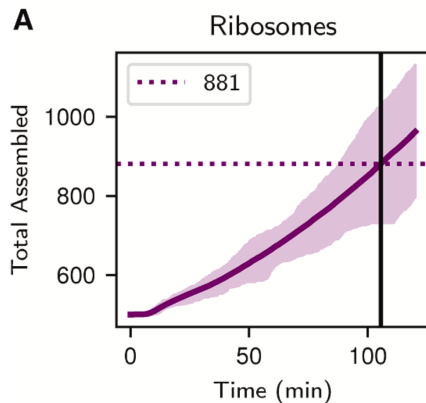
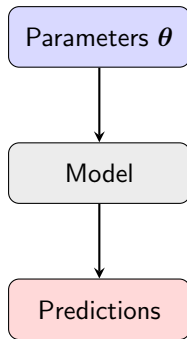


Whole-Cell Models: The State of the Art

- Representative SOTA WCM:
Luthey-Schulten Lab's 4D model (MC4D, *Cell* 2026)
 - 493 genes, 4 coupled solvers
 - 3D spatial resolution at 10 nm voxels
 - GPU-accelerated, full cell cycle



WCMs are Forward Simulators

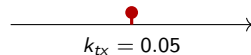


From *Thornburg et al., Cell 2026, Fig. 5*

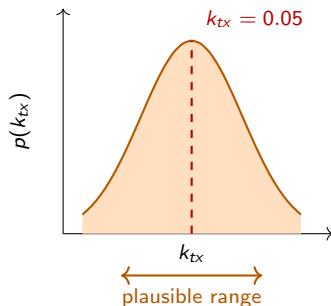
But the parameters themselves are uncertain (epistemic uncertainty)

1. Parameters are uncertain.
2. Uncertainty propagates nonlinearly.
Uncertain rates $\rightarrow$ mRNA $\rightarrow$ protein $\rightarrow$ flux $\rightarrow$ growth rate.
3. The architecture has no place to store/propagate uncertainty

What we pretend



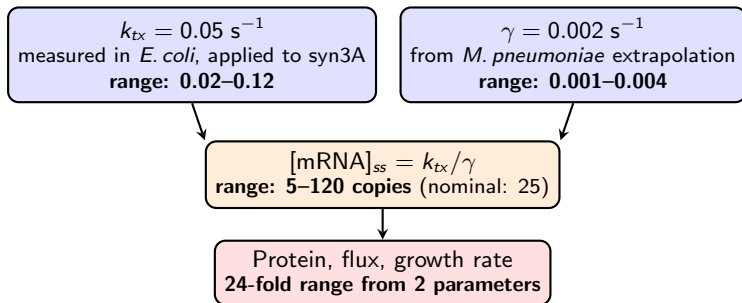
What we actually know



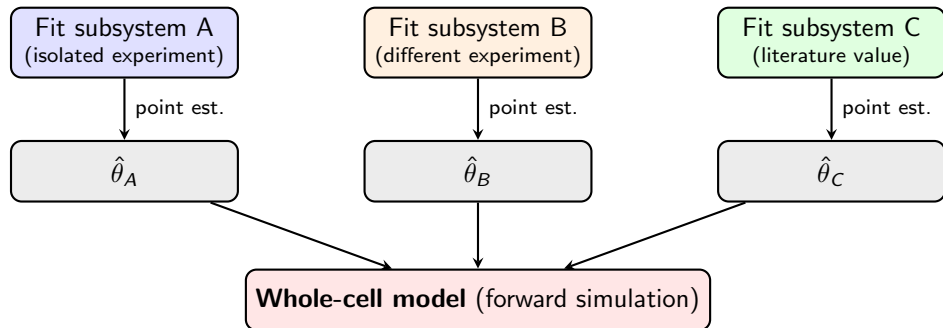
What Does That Uncertainty Look Like?

A minimal model with two parameters:

$$\frac{d[\text{mRNA}]}{dt} = k_{tx} - \gamma \cdot [\text{mRNA}] \Rightarrow [\text{mRNA}]_{ss} = k_{tx}/\gamma$$



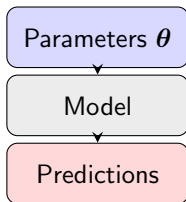
No information flows between modules



- Each subsystem calibrated in isolation — coupling is invisible during fitting
- Parameters that are jointly plausible in the coupled model may never be found

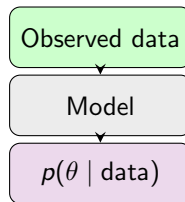
Inverting the Question

Forward simulation



"Given θ , what does the cell do?"

Inference



"Given data, what θ are consistent?"

InferCell: Design Principles

InferCell is a composable, inference-first whole-cell modelling framework in Julia.

- **Inference-first.**

UQ and probabilistic prediction built in, not bolted on.

- **Modular.**

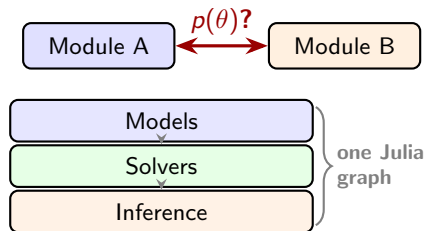
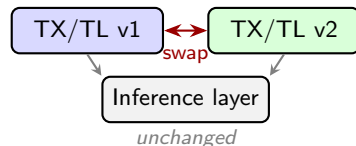
Swap modules; inference still works.

- **Information flow as design test.**

Every boundary: “how does inference cross this?”

- **Pure Julia.**

One language, one computational graph



What Becomes Possible

Questions that forward simulation cannot pose:

- Probabilistic prediction

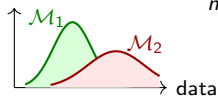
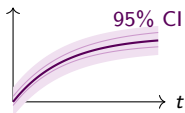
"Is doubling time 110 ± 5 min, or 60–200?"

- Parameter calibration

"What is k_{tx} for this gene, given our data?"

- Identifiability

"Does data constrain k_{tx} alone, or only k_{tx}/γ ?"



- Model selection

"Does the data prefer ODE or SSA dynamics?"

- Experimental design

"Which experiment most reduces uncertainty in γ_{prot} ?"

- Fundamental biology

"Does the data require enzyme feedback in metabolism?"

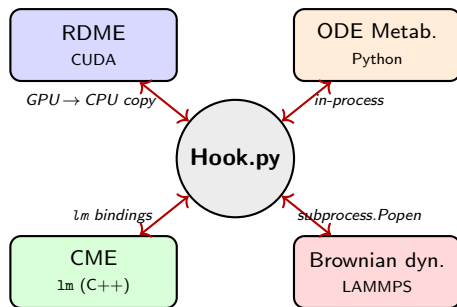


Why Pure Julia?

Standard WCMs are polyglot stacks.

- Python + C + CUDA + LAMMPS, glued by IO
- Module boundaries require re-encoding data (file IO, language bindings, GPU—CPU copy) rather than passing pointers
- Gradients don't compose across language boundaries

InferCell: one Julia stack



What Becomes Possible (Engineering)

Engineering capabilities that a single compiled runtime unlocks:

Performance & HPC

- Whole model as one program — one type system, one IR, one GPU pipeline

End-to-end gradients

- AD through solvers, models, and inference — HMC/NUTS, BOED, Fisher information

Hybrid inference

- Combine NUTS and SBI in one inference pipeline — no inter-language plumbing

Composability & substitution

- Swap solvers (ODE $\leftrightarrow$ SSA), drop in ML surrogates — no cross-language plumbing

Shared infrastructure

- One type system, one dependency stack, uniform logging and stack traces

Cheap prototyping

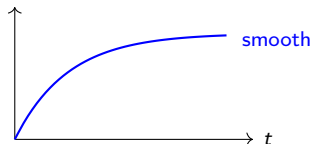
- Add a module without committing to its language's ecosystem

Hybrid Inference

Cells mix fundamentally different dynamics in the same model:

Differentiable dynamics

ODE metabolism, continuous kinetics

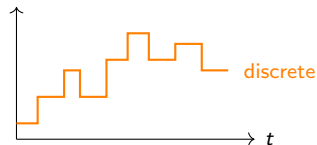


- Gradients exist
- **NUTS / HMC**

Orders of magnitude more efficient

Stochastic dynamics

Gillespie SSA, gene switching



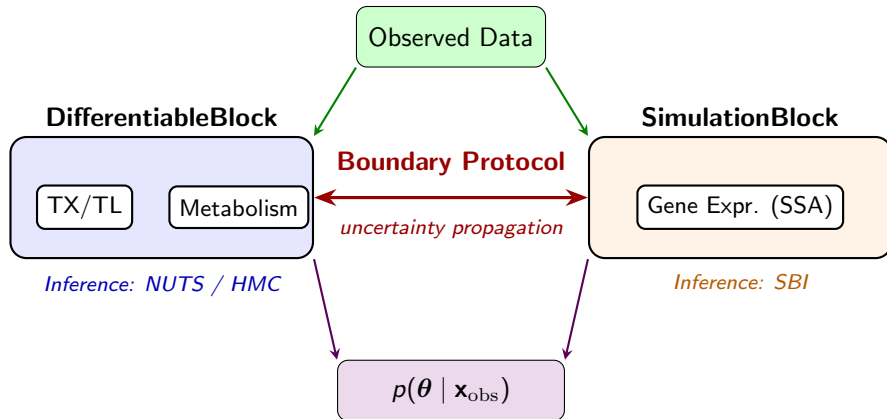
- No gradient path
- **SBI / ABC-SMC**

Handles any simulator as black box

No single inference method handles both.

The hybrid graph uses each method where it is strongest.

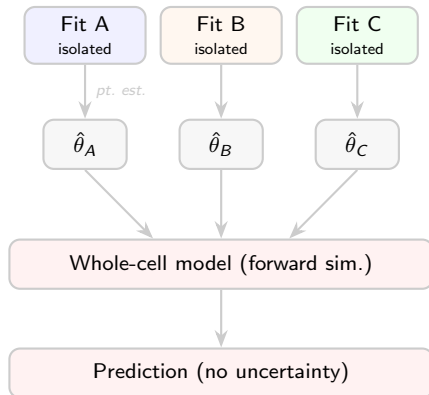
The Inference Graph



Modules are automatically partitioned into blocks by their declared inference mode.

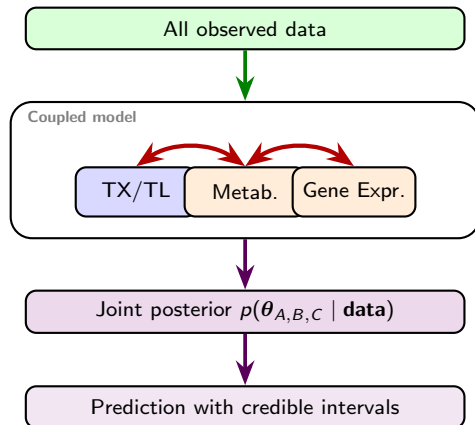
Conditioning the Whole Model

Standard approach



- ✗ uncertainty discarded at every step
- ✗ coupling invisible during fitting

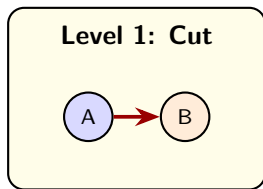
InferCell approach



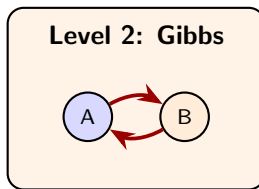
- ✓ full uncertainty propagated throughout
- ✓ coupling visible during inference

How Much Coupling Do You Need?

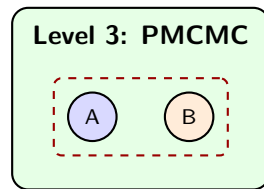
Joint inference over all parameters is intractable.
Isolated submodel fits discard all coupling.



Downstream
sees upstream
uncertainty; no feedback



Information flows both
ways; approximate



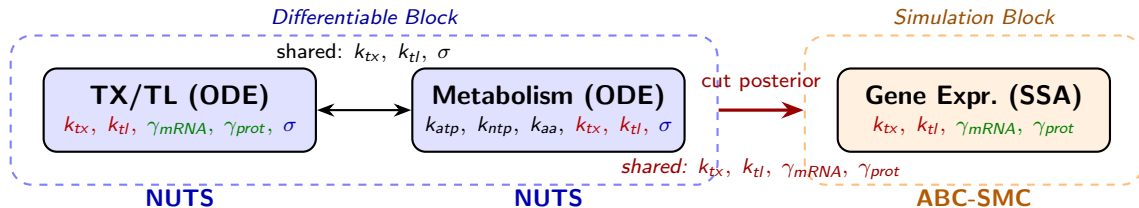
Asymptotically correct;
expensive

isolated fits

accuracy / cost $\longrightarrow$

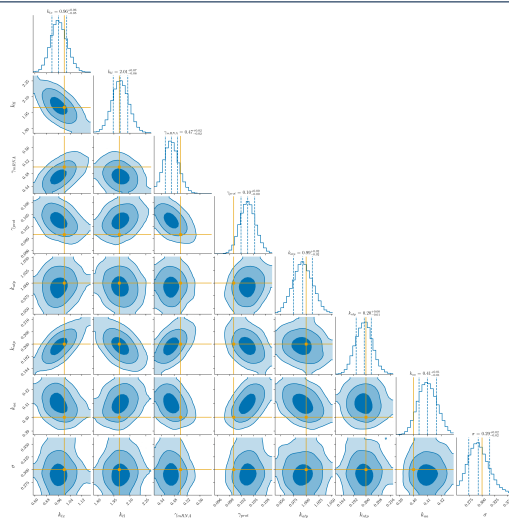
full joint

Toy example: a 3-module WCM

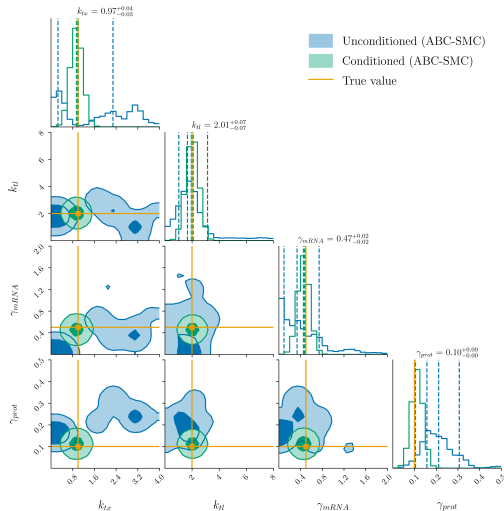


- Same biology (TX/TL) modelled as ODE and SSA $\rightarrow$ direct comparison of posteriors across formalisms
- ~ 9 unique parameters after deduplication (shared across modules), all with priors
- Validated on synthetic data where ground truth is known

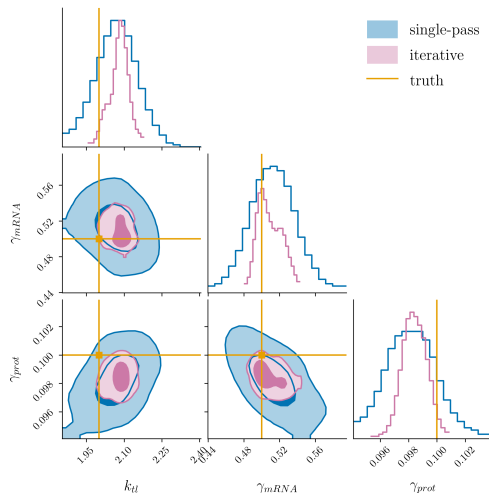
Joint parameter estimation - differentiable block



Joint parameter estimation - simulation block



Joint parameter estimation - differentiable block - Gibbs-like



Summary

1. **The problem.** Whole-cell models take ~ 1000 parameters from literature and simulate forward. No uncertainty quantification, no way to learn from data, no way to know which predictions to trust.
2. **InferCell inverts the question.** Given observations, infer parameters and their uncertainties. The model becomes something you condition on data, not just run forward.
3. **Interested?** Get involved!
 - github.com/MACSYS-CoE/InferCell
 - Please come talk to me or [open an issue](#)

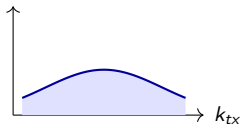
Backup

Bayesian Inference in One Slide

Toy example: given noisy mRNA measurements, what is the transcription rate k_{tx} ?

Prior $p(\theta)$

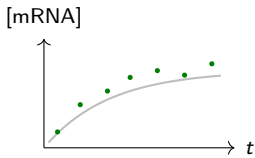
what we believe before seeing data



broad — many values plausible

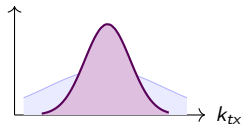
Data $p(\text{data} | \theta)$

what the experiment says



Posterior $p(\theta | \text{data})$

what to believe after data



tight — data has constrained it

$$\underbrace{p(\theta | \text{data})}_{\text{posterior}} \propto \underbrace{p(\text{data} | \theta)}_{\text{likelihood}} \cdot \underbrace{p(\theta)}_{\text{prior}}$$

noisy observations + model

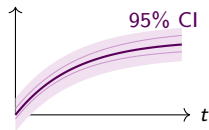
The posterior is the output. Predictions with credible intervals, identifiability, model comparison — all follow.

What Inference Enables

Inference lets you ask scientific questions that forward simulation cannot pose.

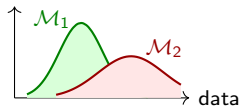
1. **Are my predictions trustworthy?**

Propagate parameter uncertainty to get credible intervals on outputs. Robust predictions vs. fragile ones.



2. **What biology can I learn?**

Bayesian model selection. Posteriors on shared parameters reveal how subsystems interact.



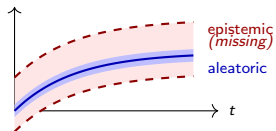
3. **What can I learn about my model?**

Which parameters are identifiable? Does the formalism matter (ODE vs SSA)? Where does the model fail to reproduce data? What experiment should you run next?



identifiable $\Rightarrow$ data-informative

MC4D: 50 replicate sims, same θ (aleatoric only)



Schematic: **parameter uncertainty** would widen the band

Implications of Root 1: No Probabilistic Information Flow (1/2)

Standard WCMs have no priors, no likelihood, no posterior — a deterministic channel from point parameters to point predictions.

Forward UQ (priors → predictions)

- No credible intervals on observables (doubling time, flux, copy numbers)
- Cannot distinguish robust predictions from fragile ones
- Cannot quantify prediction risk for downstream use (synbio design, drug targets)
- Cannot make calibrated probabilistic claims about cell behaviour

Inverse inference (data → parameters)

- No formal parameter calibration from data — only ad hoc transfer from related organisms / literature
- No joint posteriors → cannot see how parameters covary
- Cannot detect parameter “bleed-through” between modules via shared parameters
- No principled propagation of experimental uncertainty into the model
- No mechanism for Bayesian updating as new data arrives

Identifiability and structural diagnostics

- Cannot tell which parameters the data actually constrains
- Cannot detect sloppy directions in parameter space (Sethna-style sloppy models)
- Cannot detect practical vs. structural non-identifiability
- Cannot diagnose when nominally distinct parameters are degenerate

Implications of Root 1: No Probabilistic Information Flow (2/2)

Model criticism and selection

- No marginal likelihood $\rightarrow$ no Bayes factors, no principled model comparison
- Cannot compare formalisms (ODE vs SSA vs SDE) on the same data
- Cannot run posterior predictive checks to detect misspecification
- Cannot detect when literature parameter values are inconsistent with observed data
- Cannot test competing mechanistic hypotheses against each other

Experimental design

- No Bayesian optimal experimental design (requires expected information gain over posteriors)
- Cannot prioritize which parameters to measure next
- Cannot quantify the expected value of a new experiment before running it

Biology discovery

- Cannot ask “is mechanism X consistent with the data?” probabilistically
- Cannot quantify evidence for/against alternative mechanisms
- Cannot identify gaps in the model where data demands a mechanism not yet included

Communication and reproducibility

- Published predictions cannot carry honest uncertainty
- Cannot reproduce the *confidence* of a result, only point estimates

Implications of Root 2: No Unified Execution Substrate (1/2)

Standard WCMs span languages, processes, and runtimes — no single environment for the whole model.

Performance / HPC

- IO and serialization penalties at every module boundary
- Subprocess spawn-up cost (LAMMPS, LSODA, Python glue)
- Memory copies between runtimes — no pass-by-reference
- Cannot run as a single GPU kernel pipeline
- Cannot profile or optimize the whole model as one program
- Inter-language data marshalling becomes a bottleneck at scale

Differentiability and inference efficiency

- No gradient flow across language boundaries → no end-to-end AD
- Without AD, restricted to gradient-free inference (orders of magnitude more expensive)
- Cannot use HMC/NUTS, variational inference, or gradient-based optimization
- Cannot do gradient-based sensitivity analysis
- Cannot do gradient-based experimental design

Implications of Root 2: No Unified Execution Substrate (2/2)

Composability and substitution

- Cannot drop in an ML surrogate for a slow module without cross-language wrapping
- Cannot use mixed precision (reduced precision in one module, full in another)
- Cannot share a type system across modules (typed states, units, validation)
- Cannot reuse infrastructure (logging, checkpointing, error handling) uniformly
- Cannot easily try alternative formalisms (swap ODE→SSA) without reimplementation

Hybrid inference orchestration

- Cannot mix differentiable and simulation-based inference within one model
- No shared substrate for a boundary protocol between blocks
- Each inference backend lives in its own ecosystem, not composable

Software engineering and extensibility

- Build/install complexity (multiple toolchains, multiple dependency managers)
- Debugging and stack traces don't cross language boundaries
- Testing infrastructure fragmented across languages
- Reproducibility harder (multiple env specs, versions, OS-level dependencies)
- New contributors must learn the polyglot stack to add anything

Scientific prototyping

- Adding a module commits you to its language's ecosystem
- Cannot prototype new ideas cheaply — every experiment requires cross-language plumbing
- Limits who can contribute to model development

What “Crossed by Serialization” Means

A reference pass shares memory; a serialization re-encodes the data on the other side. The polyglot stack does the second at every boundary.

By reference (within one runtime)

- Pointer passed, not bytes
- Same memory, same type system, same garbage collector
- AD tape stays attached
- Distribution objects survive intact
- Cost: $O(1)$, nanoseconds

By serialization (across runtimes)

- Bytes encoded, shipped, decoded
- Fresh allocation; type information re-established
- AD tape broken at the boundary
- Distributions reduced to point values
- Cost: $O(N)$ per call

Four flavours in a typical polyglot WCM:

- **Subprocess (LAMMPS):** input file written, output file read — disk IO
- **Language bindings (1m C++):** pybind11/ctypes marshal types in shared address space
- **GPU ↔ CPU (CUDA RDME):** PCIe memory copy at every kernel boundary
- **C/Fortran wrappers (Python ODE):** “in-process” but NumPy ↔ C arrays marshalled per call

What Dies at a Module Boundary

Serialization re-encodes data and strips off everything except raw values. Three different objects survive a reference pass but die at a serialization. The polyglot stack pays each cost; a unified runtime pays none.

Object that dies	What it costs to lose	What InferCell preserves
Probability distribution	Information about θ destroyed at every boundary (data processing inequality)	Full posterior carried across modules; uncertainty propagates by construction
Derivative / AD tape	No end-to-end gradients $\Rightarrow$ no NUTS, no HMC, no gradient-based BOED	End-to-end differentiability through solvers, models, inference
Type information	Units lost, no compile-time validation, brittle cross-module contracts	Shared Julia type system; module interface is a typed contract

The unification. One root cause: serialization is destructive. One architectural answer: keep the whole model in one runtime so boundaries are pointer passes. The talk's two "roots" — probabilistic semantics and unified substrate — are aspects of the same boundary-preservation property.

Cross-Cutting: Capabilities That Need Both Roots

A few headline capabilities require *both* roots to be addressed — Root 1 makes them definable, Root 2 makes them tractable.

- **Scalable Bayesian inference on a whole-cell model**

Likelihood (Root 1) $\times$ efficient gradients (Root 2). Neither alone gets you HMC-at-scale.

- **Joint posteriors over coupled modules**

Probabilistic semantics (Root 1) $\times$ shared runtime to evaluate them (Root 2). Cross-module dependencies become visible only when both are present.

- **Gradient-based experimental design at scale**

Posterior (Root 1) $\times$ AD through the simulator (Root 2). Required for expected information gain over realistic models.

- **Hybrid inference across heterogeneous blocks**

Probabilistic boundary protocol (Root 1) $\times$ shared substrate to pass distributions across it (Root 2).
Differentiable + simulation-based blocks in one model.

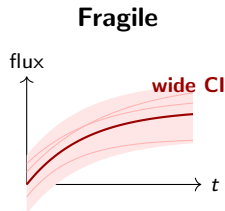
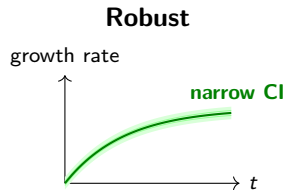
These are the capabilities that distinguish InferCell from any architecture addressing only one root.

Which Predictions Can You Trust?

Propagate the posterior through the model.
Some predictions are **robust**, others are **fragile**.

- **Robust prediction:**
Barely changes across the posterior. You can trust this output regardless of parameter uncertainty.
- **Fragile prediction:**
Swings wildly. Sensitive to parameters the data cannot constrain. This is where you need more data — or a simpler model.

Forward simulation never tells you which is which.
Every prediction looks equally confident.

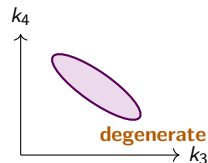
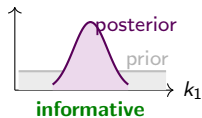


What Your Model Actually Depends On

With point estimates, every parameter looks equally important.

With posteriors, you learn **which parameters matter**:

- **Tight posterior** (narrower than prior):
Data constrains this parameter. It matters.
- **Posterior $\approx$ prior**:
Data says nothing. Redundant, or not observable.
- **Correlated posteriors**:
Parameters trade off. Neither is identifiable alone, but their combination is constrained.



These are scientific insights — they tell you what to measure next.
Forward simulation never reveals them.

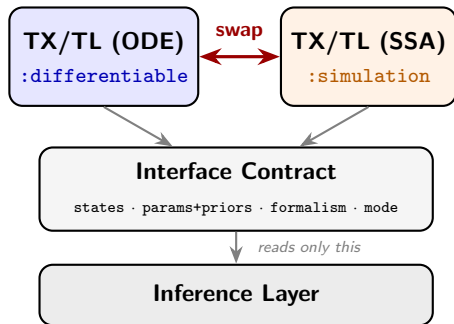
The Interface Is the Architecture

The module internals are swappable by design. What makes inference compose is the **boundary contract**:

- **States** — typed variables in, typed variables out
- **Parameters** — each carries a prior distribution
- **Formalism** — `:ode`, `:sde`, `:jump`
- **Inference mode** — `:differentiable` or `:simulation`

The inference layer sees *only* the interface. It never looks inside the module.

Design test: can you swap the internals without changing the inference call?



Modules Declare Their Inference Mode

Each module is defined by a **typed interface** that declares its dynamics *and* how inference should work:

```
abstract type AbstractSubModel end

states(m) # state variables
parameters(m) # with priors
dynamics(u, p, t, m) # rate equations
formalism(m) # :ode, :sde, :jump
inference_mode(m) # :differentiable,
                  # :simulation, :auto
```

- The framework reads these declarations and **automatically partitions** modules into the right inference backend
- Adding a module extends the inference problem **without code changes** to the inference layer
- Swap an ODE model for an SSA model — the backend switches automatically

Modularity at the inference level, not just the simulation level.

Parameters Carry Priors

Every parameter declares its value, prior distribution, and whether it is free or fixed:

```
struct InferParameter
  value::Float64
  prior::Distribution
  fixed::Bool
  name::Symbol
  role::Symbol # :rate, :initial,
               # :observation
end
```

- **Fixed:** point estimate from literature
- **Constrained:** informative prior from experiment
- **Free:** to be inferred from data
- The inference layer reads these declarations and builds the Bayesian model automatically
- Sensitivity analysis is a flag toggle, not a code change

Boundary Protocol: Three Levels

1. **Level 1 — Sequential conditioning (v1 target)**

Infer differentiable block $\rightarrow$ posterior samples $\rightarrow$ condition simulation block on those samples.

One-directional. Produces a cut posterior. Valid under conditional independence.

2. **Level 2 — Gibbs-like alternation (v1 demonstration)**

Alternate: fix stochastic module state $\rightarrow$ NUTS on differentiable params; fix differentiable params $\rightarrow$ SBI on stochastic params.

Bidirectional. Pseudo-marginal Gibbs sampler. Convergence depends on ABC tolerance.

3. **Level 3 — Particle MCMC (future work)**

Stochastic modules contribute unbiased likelihood estimates via particle filters into a joint MCMC sampler.

Asymptotically exact. Computationally expensive.

End-to-End Workflow

```
using InferCell

# 1. Define sub-model
txl = TranscriptionTranslation()

# 2. Build problem (orchestrator composes modules)
prob = build_problem([txl]; tspan=(0.0, 100.0))

# 3. Generate synthetic data
true_sol = solve(prob, Tsit5())
data = observe(true_sol, 0.0:5.0:100.0; sigma=0.1)

# 4. Infer parameters (NUTS via Turing.jl)
chain = infer(txl, data)

# 5. Posterior predictive check
ppc = posterior_predictive(txl, chain)
```

Same `infer()` call works for a single module or a composed multi-module model.

Neural Surrogates: Differentiable by Construction

An ML surrogate replaces an expensive module with a learned function. Because Julia's ML stack (Flux.jl, Lux.jl) shares the AD substrate, gradients flow through surrogates end-to-end — when the surrogate lives in the same runtime.

Differentiable surrogate types

- Neural networks (MLP, neural ODE) — Flux.jl, Lux.jl
- Normalising flows — Bijectors.jl
- Gaussian processes — AbstractGPs.jl

Non-differentiable surrogates

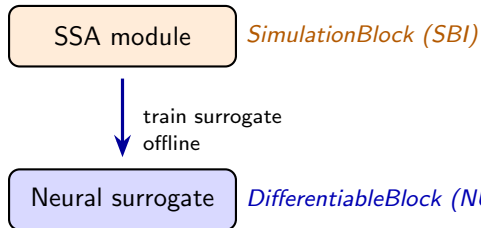
- Trees, argmax outputs, non-reparameterised samplers
- Slot into the SimulationBlock with SBI — the hybrid architecture already handles this case

The catch — same AD substrate

- Surrogates are only differentiable end-to-end if they live in the same AD universe as the rest of the model
- A PyTorch surrogate plus a Julia model is still a polyglot stack
- Gradients die at the language boundary, same as for any other serialised module
- Julia's ML ecosystem (Flux, Lux) integrates with Zygote/Enzyme transparently; PyTorch-via-PyCall does not

Surrogates Move Modules Between Blocks

The architectural lever: a differentiable surrogate trained on a stochastic simulator's outputs can move a module from SimulationBlock to DifferentiableBlock.



Recipe

1. Run slow simulator offline; collect (θ, y) pairs
2. Train a differentiable surrogate (e.g. normalising flow over outputs)
3. Replace the simulator module with the surrogate
4. Module migrates to the DifferentiableBlock; NUTS now handles it

Payoff

- Orders-of-magnitude speedup at inference time
- End-to-end gradients across a previously stochastic module
- Inference graph adapts to module changes — not static

TX/TL Module: Equations & Priors

Two-state ODE per gene; four rate parameters plus observation noise:

$$\frac{d[\text{mRNA}]}{dt} = k_{tx} - \gamma_{mRNA} \cdot [\text{mRNA}]$$

$$\frac{d[\text{protein}]}{dt} = k_{tl} \cdot [\text{mRNA}] - \gamma_{prot} \cdot [\text{protein}]$$

Parameter	Description	Prior
k_{tx}	Transcription rate	LogNormal(0, 1)
k_{tl}	Translation rate	LogNormal(0, 1)
γ_{mRNA}	mRNA degradation rate	LogNormal(0, 1)
γ_{prot}	Protein degradation rate	LogNormal(0, 1)
σ_{obs}	Observation noise	Normal(0, 1) truncated ≥ 0

Initial conditions mRNA_0 , prot_0 declared with Normal(0, 1) priors, flagged fixed by default (toggleable).

Multi-gene variant: per-gene rates and ICs; one shared σ_{obs} .

Metabolism Module: Equations & Priors

Three-state ODE (ATP, NTP, AA) with enzyme-modulated ATP production. Inputs from TX/TL: mRNA, enzyme-protein E .

$$\frac{d[\text{ATP}]}{dt} = k_{atp} \left(1 + \frac{E}{K_M + E} \right) (\text{ATP}_{\max} - \text{ATP}) - e_{tx} k_{tx} - e_{tl} k_{tl} [\text{mRNA}] - k_{ntp} \text{ATP} - k_{aa} \text{ATP}$$

$$\frac{d[\text{NTP}]}{dt} = k_{ntp} \text{ATP} - n_{tx} k_{tx} - \gamma_{ntp} \text{NTP}$$

$$\frac{d[\text{AA}]}{dt} = k_{aa} \text{ATP} - n_{tl} k_{tl} [\text{mRNA}] - \gamma_{aa} \text{AA}$$

Free parameter	Description	Prior
k_{atp}, k_{ntp}, k_{aa}	ATP/NTP/AA synthesis rates	LogNormal(0, 1)
k_{tx}^*, k_{tl}^*	Shared with TX/TL (joint NUTS)	LogNormal(0, 1)
σ_{obs}	Observation noise	Normal(0, 1) truncated ≥ 0

ICs (Normal($\cdot$, 1), fixed by default): $\text{ATP}_0=5$, $\text{NTP}_0=2$, $\text{AA}_0=2$.

Fixed structural constants: $\text{ATP}_{\max}=10$, $e_{tx}=0.1$, $e_{tl}=0.05$, $n_{tx}=n_{tl}=0.5$, $\gamma_{ntp}=\gamma_{aa}=0.1$, $K_M=5$.

* Shared parameters appear in both TX/TL and Metabolism — joint NUTS infers a single posterior.

Bursty SSA Module: Reactions & Priors

Telegraph-promoter gene expression as a jump process. States: (promoter $\in \{0, 1\}$, mRNA, protein).

Reaction network (6 ConstantRateJumps):

(1) $\emptyset \rightarrow \text{promoter ON}$	rate: $k_{on} (1 - \text{promoter})$
(2) $\text{promoter ON} \rightarrow \emptyset$	rate: $k_{off} \cdot \text{promoter}$
(3) $\emptyset \rightarrow \text{mRNA}$	rate: $k_{tx,burst} \cdot \text{promoter}$
(4) $\text{mRNA} \rightarrow \emptyset$	rate: $\gamma_{mRNA} \cdot \text{mRNA}$
(5) $\emptyset \rightarrow \text{protein}$	rate: $k_{tl} \cdot \text{mRNA}$
(6) $\text{protein} \rightarrow \emptyset$	rate: $\gamma_{prot} \cdot \text{protein}$

Free parameter	Description	Prior
k_{on}, k_{off}	Promoter switching rates	LogNormal(0, 1)
$k_{tx,burst}$	Burst transcription rate	LogNormal(0, 1)
$k_{tl}^*, \gamma_{mRNA}^*, \gamma_{prot}^*$	Shared with TX/TL (cross-formalism)	LogNormal(0, 1)

ICs (Normal(0, 1), fixed by default): promoter₀, mRNA₀, protein₀.

Inference: ABC-SMC on summary statistics — no explicit likelihood. * shared params receive conditioned priors from the DifferentiableBlock posterior via the boundary protocol.

v0.0.1: Implementation Status (May 2026)

Modules

- TX/TL (ODE), multi-gene
- Metabolism (ODE), + enzyme feedback
- Bursty SSA, telegraph promoter

Block inference

- DifferentiableBlock $\rightarrow$ joint NUTS
- SimulationBlock $\rightarrow$ ABC-SMC

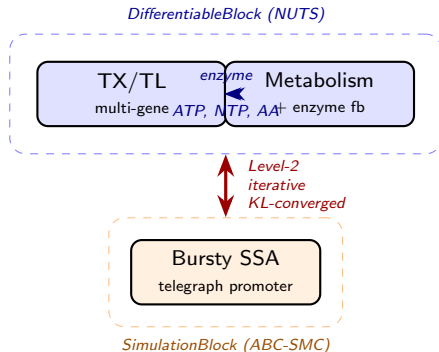
Boundary protocol

- Level 1 (cut posterior) ✓
- Level 2 (iterative, KL-converged) ✓

Robustness

- Tier B mismatch generator

Integration test (Slurm, ~32 min): Level-2 narrows shared-param 90% CIs by >50% vs single-pass.



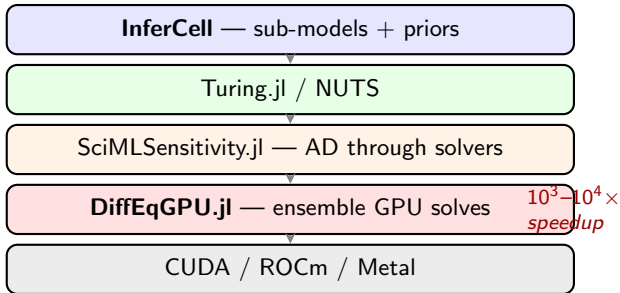
GPU Acceleration and the Julia Advantage

The bottleneck: every NUTS leapfrog step solves the full ODE system. As models grow, this means $O(10^4)$ ODE solves per chain.

Julia's single-language stack:

- **DiffEqGPU.jl:** batch ODE solves on GPU — same code, one-line backend switch
- **AD composes with GPU kernels:** gradients through GPU-accelerated solvers, no custom CUDA
- **KernelAbstractions.jl:** write once, run on CUDA, ROCm, Metal, or CPU

No serialisation between sampler $\leftrightarrow$ solver $\leftrightarrow$ GPU. One compiled representation top to bottom.



What this enables:

- Parallel MCMC chains across GPU cores
- Posterior predictive checks batched on GPU
- Adjoint sensitivity methods scale to 100+ parameters without linear cost growth

Why Julia (and Not JAX)?

Julia wins on composability:

- DiffEq + Turing + SciMLSensitivity + Catalyst share one type system — AD flows end-to-end with no language boundaries
- Multiple dispatch makes the `AbstractSubModel` interface natural: `formalism(m)` dispatches to the right solver
- Catalyst.jl gives a CRN DSL with no JAX equivalent — critical for structural inference

For v1, Julia is OK:

The research contribution is the architecture and boundary protocol, not the scale. Julia lets us express the ideas most directly.

JAX would be ahead on:

- **SBI tooling.** Python `sbi` library is more mature (neural density estimators, normalizing flows)
- **GPU at scale.** XLA compilation, `vmap/pmap` over samples — smoother path to 100+ parameters
- **Community.** Larger pool of computational biology contributors
- **ML integration.** Neural surrogates, amortized inference live in PyTorch/JAX

The risk to watch:

If InferCell needs neural SBI or heavy GPU batching at whole-cell scale, the ecosystem gap could be annoying.

Beyond Parameter Uncertainty: Which Reactions Exist?

So far: **fixed model structure**, infer rate constants.

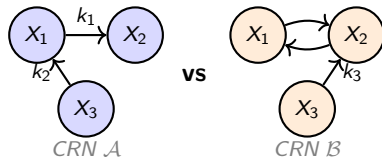
But often the structure itself is uncertain:

- Is gene X regulated by protein Y ?
- Does this metabolite inhibit that enzyme?
- Which of several proposed pathways is active?

Different CRN structures can fit training data equally well but **diverge catastrophically under novel conditions** (Foo et al. 2025, Fig. 2: reconstruction error small, prediction error $10\times$ larger for wrong structure).

Structural uncertainty can dominate parameter uncertainty.

Which CRN?



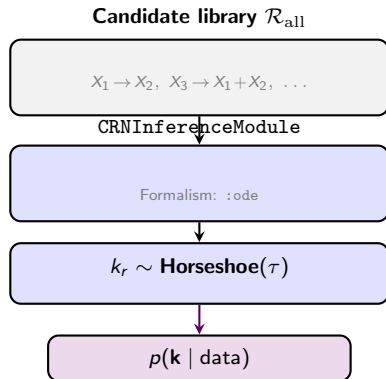
Both fit the data.
Only one predicts correctly
under new conditions.

CRN Inference as a Plug-and-Play Module

Key idea: include *all* candidate reactions in one module. Let the **posterior** decide which are present.

- Define a candidate library $\mathcal{R}_{\text{all}}$
- Dynamics: $\dot{x}_s = \sum_r \nu_{sr} k_r \prod_j x_j^{m_{rj}}$
- One rate constant k_r per reaction
- Horseshoe prior: $k_r \sim \text{HS}(\tau)$
Pushes most $k_r \rightarrow 0$
- Posterior over $\mathbf{k}$ is the posterior over structures

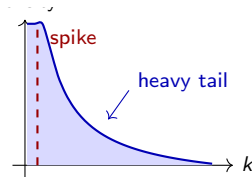
No new inference machinery needed.
Same `infer()` call. The prior does the work.



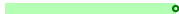
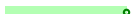
How the Horseshoe Prior Induces Structure

The **horseshoe prior** $k_r \mid \lambda_r \sim \mathcal{N}^+(0, \lambda_r^2 \tau^2)$, $\lambda_r \sim \text{C}^+(0, 1)$ has two key properties:

1. **Spike at zero.** Most rate constants are shrunk to effectively zero $\rightarrow$ those reactions are “absent.”
2. **Heavy tails.** Reactions that *are* present retain their full magnitude — minimal bias on the non-zero rates.



Posterior for 5 candidate reactions

k_1		present
k_2		
k_3		absent
k_4		
k_5		uncertain

The posterior naturally separates into:

- $k_r \approx 0$: reaction r absent
- $k_r \gg 0$: reaction r present

Reaction inclusion probability:

$$P(r \in \mathcal{R} \mid \text{data}) = P(k_r > \epsilon \mid \text{data})$$

Relation to Foo et al. (2025): Structural Uncertainty in CRNs

Their approach:

1. Sparse optimisation (BFGS) with penalty functions (L1, horseshoe, etc.)
2. Multiple restarts $\times$ multiple λ values
3. Keep all local optima $\rightarrow$ map to CRN structures
4. BIC approximation for model evidence
5. Approximate posterior over CRN structures

Key results:

- Recovers dynamically equivalent CRNs
- Identifies structural ambiguities via posterior correlations
- Hierarchical tree of alternative pathways

Foo, Zanca, Flegg & Siekmann (2025). *Quantifying structural uncertainty in chemical reaction network inference*. arXiv:2505.15653.

Can InferCell do this natively?

- Penalty functions $\rightarrow$ **horseshoe prior**
Same sparsity, fully Bayesian
- Multi-start optimisation $\rightarrow$ **NUTS sampling**
Explores multimodal posterior directly
- BIC approximation $\rightarrow$ **exact posterior**
No need for separate model evidence step
- Separate pruning/recombination pipeline $\rightarrow$ **not needed**
Posterior samples directly encode structure

Composing Structural Inference with the Inference Graph

Structural inference plugs into the full InferCell architecture:

- **Stage 1:** Discover which reactions are active from data (CRN inference module)
- **Stage 2:** Full Bayesian inference on the discovered structure, composed with other modules
- Or run both simultaneously — horseshoe parameters live alongside standard rate constants in the same NUTS sampler

